

INSTRUCCIONES GENERALES PARA TODO EL EQUIPO

Reglas del proyecto (OBLIGATORIAS)

Todos los desarrolladores deben cumplir estas reglas:

1. Arquitectura de microservicios.
2. El backend debe desarrollarse con **Python + FastAPI** y el frontend con **React**, según la guía del proyecto.
3. Cada microservicio debe tener su propia base de datos.
4. Ningún servicio puede leer directamente la base de datos de otro servicio.
5. La comunicación entre servicios debe realizarse por API REST y contratos definidos.
6. El frontend solo se comunica con el BFF/API Gateway.
7. El frontend NO tiene lógica de negocio.
8. El BFF NO tiene lógica de negocio; solo enruta y adapta.
9. Cada módulo debe ser independiente.
10. Cada desarrollador debe entregar:
 - Código funcionando
 - Endpoints o pantallas funcionando
 - Base de datos funcionando
 - Documento técnico
 - Evidencia (capturas o video) las cuales serán cargadas en la presentación
11. Todo debe correr con Docker al final del proyecto.
12. Arquitectura interna por servicio: routers, application, domain, infrastructure.
13. La persistencia debe construirse desde el propio microservicio; no se deben usar scripts SQL manuales como base principal del diseño.
14. Cada desarrollador debe documentar su componente y su forma de ejecución.

QUÉ DEBE ENTREGAR CADA DESARROLLADOR (FORMATO)

Cada developer debe entregar:

1. Código del microservicio o frontend.
2. Base de datos.
3. Endpoints funcionando.

4. Documento técnico:
 - Propósito del módulo
 - Endpoints
 - Modelo de datos
 - Decisiones técnicas
 - Cómo ejecutar
 - Limitaciones
5. Evidencia:
 - Capturas de Postman / Frontend
6. Dockerfile del servicio (Sprint 3 obligatorio).

DEV 1 — AUTH / SEGURIDAD

Historia de Usuario

Como usuario del sistema, quiero autenticarme de forma segura para acceder únicamente a las funcionalidades autorizadas.

Requisitos

- El sistema debe permitir login.
- El sistema debe generar un token JWT.
- El sistema debe validar el token en endpoints protegidos.
- Debe existir manejo de error si el login falla.
- La autenticación debe estar desacoplada del frontend.

Criterios de Aceptación

- Si el usuario envía credenciales correctas, el sistema genera un JWT.
- Si el usuario envía credenciales incorrectas, el sistema responde con error.
- Si se intenta acceder a un endpoint protegido sin token, el sistema responde 401.
- Si el token es inválido o expirado, el sistema responde 401.
- El servicio funciona de manera independiente.

Actividades

- Crear microservicio de autenticación.
- Crear endpoint /login.
- Implementar generación de JWT.
- Implementar validación de JWT.
- Proteger endpoints.
- Documentar endpoints.

Entregable

Servicio de autenticación funcional y documentado.

DEV 2 — INVENTORY SERVICE

Historia de Usuario

Como administrador, quiero cargar archivos de inventario para registrar productos en lote dentro del sistema.

Requisitos

- El sistema debe permitir carga de archivos.
- Debe validar estructura del archivo.
- Debe registrar los productos válidos.
- Debe registrar errores por fila.
- Debe existir trazabilidad por lote.

Criterios de Aceptación

- Cada carga genera un ImportBatch.
- El sistema registra filas válidas e inválidas.
- Los errores se almacenan con número de fila y descripción.
- Los productos válidos se guardan en la base de datos.
- Si el libro es usado, debe registrar condición.
- Si tiene defectos, deben describirse.
- Se registran processed_rows, valid_rows, invalid_rows.

Actividades

- Crear entidades InventoryItem, ImportBatch, ImportError.
- Crear endpoint de carga masiva.
- Validar archivo.
- Guardar datos válidos.
- Guardar errores.
- Documentar endpoint.

Entregable

Microservicio de inventario operativo con carga masiva y trazabilidad por lote.

DEV 3 — CATALOG SERVICE

Historia de Usuario

Como administrador, quiero registrar y consultar productos bibliográficos para disponer de un catálogo base navegable y estructurado.

Requisitos

- Registrar libros.
- Consultar libros.
- Persistencia propia.
- Manejo de categorías.
- Manejo de ISBN/ISSN si existen.

Criterios de Aceptación

- Se pueden crear libros.
- Se pueden consultar libros.
- El catálogo tiene su propia base de datos.
- No se puede crear libro sin información mínima.
- No se puede crear un libro sin título y autor.
- El servicio expone endpoints de consulta.

Actividades

- Crear entidad Book.
- Crear entidad Category.
- Crear endpoints POST /books y GET /books.
- Conectar a PostgreSQL.
- Documentar endpoints.

Entregable

Microservicio de catálogo funcional con persistencia propia.

DEV 4 — FRONTEND ADMIN

Historia de Usuario

Como administrador, quiero visualizar el inventario cargado y su estado para poder operar y revisar la información desde una interfaz.

Requisitos

- Debe existir interfaz administrativa.
- Debe mostrar inventario o lotes cargados.
- Debe consumir backend.
- No debe tener lógica de negocio.

Criterios de Aceptación

- Existe pantalla de administración.
- Se visualiza inventario cargado.
- Se visualizan estados (cargado, error, vacío).
- El frontend consume API, no lógica interna.
- Se visualiza el estado del lote (procesado, con errores, completado).
- Se visualiza cantidad de registros válidos e inválidos.

Actividades

- Crear proyecto React.
- Crear pantalla admin.
- Consumir API inventario.
- Mostrar tabla.
- Manejar loading y error.

Entregable

Panel administrativo inicial en React conectado al backend.

DEV 5 — FRONTEND COMERCIAL

Historia de Usuario

Como usuario, quiero visualizar el catálogo base para consultar libros disponibles desde una interfaz simple.

Requisitos

- Debe existir catálogo web.
- Debe listar libros.
- Debe consumir backend.
- Debe permitir navegación básica.

Criterios de Aceptación

- Se visualiza listado de libros.
- La información viene del backend.
- Se puede navegar el catálogo.
- El catálogo muestra título del libro, autor y categoría.
- Maneja loading y errores.

Actividades

- Crear página catálogo.
- Consumir API catálogo.
- Mostrar listado.
- Crear vista básica.

Entregable

Catálogo web inicial funcional en React.

DEV 6 — AI ENRICHMENT SERVICE (MOCK)

Historia de Usuario

Como sistema, quiero preparar el servicio de enriquecimiento para desacoplar desde el inicio la lógica de IA del resto del backend.

Requisitos

- Debe existir microservicio independiente.
- Debe existir endpoint de enriquecimiento.
- Debe devolver respuesta mock.
- Debe quedar listo para conectar IA real en Sprint 2.

Criterios de Aceptación

- El servicio existe y corre independiente.
- El endpoint recibe datos del libro.
- El endpoint devuelve datos enriquecidos mock.
- La respuesta mock debe incluir descripción, categoría y palabras clave.
- La estructura permite integrar APIs externas después.

Actividades

- Crear microservicio.
- Crear endpoint /enrich.
- Crear respuesta mock.
- Documentar contrato.

Entregable

Servicio de enriquecimiento base listo para integración posterior.

DEV 7 — DATA QUALITY / ERRORES

Historia de Usuario

Como administrador, quiero consultar los errores de carga del inventario para identificar problemas y corregirlos

Requisitos

- Debe registrar errores por fila.
- Debe permitir consultar errores.
- Debe relacionar error con lote.

Criterios de Aceptación

- Se pueden consultar errores por lote.
- Cada error muestra fila y descripción.
- Los errores quedan almacenados.
- Se puede consultar errores por batch_id.
- Permite identificar problemas de calidad de datos.

Actividades

- Crear endpoint consulta errores.
- Crear estructura ImportError.
- Probar con archivo inválido.

Entregable

Módulo de calidad de datos y errores operando sobre la carga de inventario.

DEV 8 — CONFIGURACIÓN DEL SISTEMA

Historia de Usuario

Como administrador, quiero definir parámetros básicos del sistema para controlar comportamientos generales sin cambiar el código.

Requisitos

- Debe existir módulo de configuración.
- Debe permitir consultar parámetros.
- Debe permitir cambiar parámetros sin cambiar código.

Criterios de Aceptación

- El sistema puede leer parámetros.
- Los parámetros afectan comportamiento del sistema.
- El módulo es reutilizable.

Actividades

- Crear entidad configuración.
- Crear endpoint de consulta.
- Integrar parámetro en otro módulo.

Entregable

Módulo de configuración funcional y reutilizable.

Ejemplos:

- porcentaje_descuento_base
- porcentaje_comision
- limite_stock_minimo

DEV 9 — BFF / API GATEWAY

Historia de Usuario

Como sistema, quiero centralizar el acceso del frontend a los microservicios para simplificar contratos y desacoplar la interfaz del backend interno.

Requisitos

- Debe existir API Gateway.
- Debe enrutar a microservicios.
- Debe ser punto único de entrada.
- No debe contener lógica de negocio.

Criterios de Aceptación

- El frontend consume servicios a través del gateway.
- El gateway enruta correctamente.
- Maneja errores.
- Centraliza acceso.
- El frontend solo consume endpoints del BFF.
- El BFF enruta a auth, inventory y catalog.

Actividades

- Crear API Gateway.
- Conectar auth, inventory y catalog.
- Crear rutas.
- Probar conexión.

Entregable

API Gateway / BFF funcional conectado a los primeros microservicios.

ENTREGABLES DETALLADOS POR CADA DESARROLLADOR

SPRINT 1 — ENTREGABLES POR DESARROLLADOR

DEV 1 — AUTH SERVICE

Debe entregar:

- Microservicio Auth funcionando.
- Endpoint POST /login.
- Generación de JWT.
- Validación de JWT.
- Endpoint protegido de prueba.
- Base de datos auth_db.

Debe demostrar:

- Login correcto → devuelve token.
- Login incorrecto → error.
- Endpoint sin token → 401.
- Endpoint con token → OK.

Debe documentar:

- Qué hace el servicio.
 - Endpoints.
 - Estructura del proyecto.
 - Cómo ejecutarlo.
 - Decisiones técnicas.
-

DEV 2 — INVENTORY SERVICE

Debe entregar:

- Microservicio Inventory.
- Endpoint carga masiva.
- Entidades:
 - InventoryItem
 - ImportBatch

- ImportError
- Base de datos inventory_db.

Debe demostrar:

- Carga archivo.
- Registra lote.
- Guarda productos válidos.
- Guarda errores por fila.
- Muestra resumen de carga.

Debe documentar:

- Formato del archivo.
 - Endpoint.
 - Estructura BD.
 - Cómo ejecutarlo.
-

DEV 3 — CATALOG SERVICE

Debe entregar:

- Microservicio Catalog.
- Entidades Book y Category.
- Endpoints:
 - POST /books
 - GET /books
- Base de datos catalog_db.

Debe demostrar:

- Crear libro.
- Consultar libros.
- Persistencia funcionando.

Debe documentar:

- Modelo de datos.
- Endpoints.
- Cómo ejecutar.

DEV 4 — FRONTEND ADMIN

Debe entregar:

- Frontend React admin.
- Pantalla para ver inventario o lotes.
- Conexión al backend.

Debe demostrar:

- Visualización de inventario.
- Estados: loading, error, vacío.

Debe documentar:

- Componentes.
- Cómo ejecutar React.
- Cómo se conecta al backend.

DEV 5 — FRONTEND COMERCIAL

Debe entregar:

- Frontend React catálogo.
- Listado de libros.
- Conexión al backend.

Debe demostrar:

- Ver catálogo.
- Navegación básica.

Debe documentar:

- Componentes.
- Cómo ejecutar.

DEV 6 — AI ENRICHMENT SERVICE (MOCK)

Debe entregar:

- Microservicio AI Enrichment.
- Endpoint /enrich.

- Respuesta mock.
- Base de datos enrichment_db (si guarda info).

Debe demostrar:

- Enviar libro → devuelve metadata fake.

Debe documentar:

- Cómo luego se conectará a IA real.
-

DEV 7 — DATA QUALITY SERVICE

Debe entregar:

- Endpoint consulta errores.
- Relación con ImportBatch.
- Base de datos quality_db (o misma inventory).

Debe demostrar:

- Consultar errores por lote.

Debe documentar:

- Tipos de errores.
 - Estructura.
-

DEV 8 — CONFIG SERVICE

Debe entregar:

- Microservicio configuración.
- Endpoint para consultar parámetros.
- Base de datos config_db.

Debe demostrar:

- Cambiar parámetro y que afecte el sistema.

Debe documentar:

- Parámetros del sistema.
-

DEV 9 — API GATEWAY / BFF

Debe entregar:

- API Gateway.
- Rutas:
 - /auth
 - /catalog
 - /inventory
- Conexión con frontend.

Debe demostrar:

- Frontend → Gateway → Microservicio.

Debe documentar:

- Rutas.
- Cómo enruta.